COLLEGE OF COMPUTING
DEPARTMENT OF COMPUTER SCIENCE
AND ARTIFICIAL INTELLIGENCE

جامعة أم القرى
UMM AL-QURA UNIVERSITY

PROJECT REPORT: Semantic Book Recommendations System

STUDENT:

Amjad Alghreeb Alamri

AI3104 -AI System Design Project
1st semester 2025 -1447

Table of Contents

Chapter 1: Introduction

1.1 Abstract

Due to the exponential growth in the number of published books in the digital age, readers are finding it increasingly challenging to find literature that reflects their interests and underlying semantic preferences. Traditional recommendation systems, based largely on collaborative filtering or simple metadata matching, often fail to capture deeper meaning, thematic resonance, or nuanced emotional tone.

This project proposes a **web-based semantic book recommender system** that leverages specialized language models and embedding-based similarity to suggest books based on meaning, sentiment, and conceptual alignment, in addition to genre or popularity. The core goal is to design and implement a recommender engine that, given a user's natural language query (e.g., "a story of forgiveness"), returns books whose narrative semantics match that query. The system operates as a **real-time cloud service** (or **web-based service**) to ensure high availability and responsiveness, contrasting with deployment models requiring on-device or edge processing.

Key contributions include improved user experience: The system will provide more accurate recommendations relevant to the user's real interests, giving them a natural and intuitive search experience, as if they were asking a book expert. In addition, practical application of Artificial Intelligence technologies: The project is a practical and tangible application of advanced technologies, such as embedding models and vector databases.

Chapter 2: AI Lifecycle

2.1 Introduction & Problem Definition

2.1.1 Problem Statement & Background

Currently, most book recommendation systems rely on superficial factors such as general categories or user ratings, making the recommendations limited and inaccurate. This makes it difficult for readers to find books they prefer and that suit them. This project focuses on building a book recommendation system based on understanding the meaning and textual context of book descriptions using large language models (LLMs) and semantic embeddings. The system will classify and analyses book descriptions according to semantic content and literary category to provide more accurate recommendations.

2.1.2 Objectives & Requirements

2.1.2.1 Business Objectives

The primary business goal of this project is to improve the efficiency and effectiveness of book discovery and recommendation in digital reading platforms. The system aims to increase user engagement, satisfaction, and overall sales conversion through more meaningful and personalized suggestions.

Specific Business Objectives:

- Increase book sales by 20% through personalized semantic recommendations that better align with readers' interests.
- Enhance user retention rate by 15% by providing contextually relevant and book suggestions.
- Reduce user's average book search time by 30%, allowing readers to discover new books faster.
- Improve overall user satisfaction (measured via feedback surveys) to at least 85% positive Ratings.
- Support business decisions by providing insights into user reading preferences and emerging book trends.

2.1.2.2 Model Objectives

From a technical perspective, the goal is to design and implement an intelligent Semantic Book Recommender System that enhances the book discovery experience by analyzing the *meaning* and *context* of book descriptions instead of relying on ratings or categories.

Specific Model Objectives:

- Build a recommendation engine using pre-trained language models and semantic embeddings to match books with user queries based on meaning and emotional tone.
- Develop a lightweight and interactive interface for users to enter natural-language queries and instantly receive recommendations.
- Evaluate model performance using classification metrics, including Accuracy, Precision, Recall, and F1-Score, to assess the effectiveness of the emotion detection and recommendation components in classifying user intent and emotional tone.
- Deploy the system using open-source frameworks and ensure data quality, ethical use, and fairness in recommendations.

2.1.2.3 System Requirements

Functional:

- Accept user queries and generate book recommendations using semantic similarity.
- Display results with book titles, authors, and summaries.

Non-Functional:

• **Performance:**
The system shall provide real-time recommendations with an average response time of **≤ 2 seconds** under normal operating conditions.
• **Model Quality:**
The recommendation pipeline shall achieve an **F1-Score ≥ 0.8**, ensuring balanced precision and recall for semantic matching and classification tasks.
• **Scalability:**
The system shall support **at least 10,000 book records** and be scalable through horizontal scaling of the vector database and backend services. Approximate Nearest Neighbor (ANN) indexing and batch embedding generation shall be used to maintain performance as data volume increases.
• **Reliability:**
The system shall ensure high availability and stable operation by using persistent storage for embeddings, regular backups, and well-tested pre-trained models. Graceful degradation mechanisms shall be implemented so that partial functionality (e.g., recommendations without emotion filtering) remains available in case of component failure.
• **Security:**
The system shall protect user data and system resources through secure API access, input validation, and controlled access to the vector database. Data shall be protected during transmission and storage, and only trusted, official pre-trained model checkpoints shall be used.

• **Privacy and Ethical Compliance:**
The system shall comply with ethical AI principles and data privacy regulations by avoiding the storage of personally identifiable user information, ensuring transparency in recommendation logic, and minimizing bias in model outputs in alignment with SDAIA AI ethics guidelines[12].

2.1.3 Target Users

The Semantic Book Recommender is designed for the following users:

- Avid readers and bibliophiles who wish to discover books aligned with a conceptual or emotional interest (rather than simply genre).
- Book clubs or educators seeking thematically coherent reading suggestions for class discussions or reading groups.
- Casual readers who may enter a descriptive query (e.g., "a story about a dreamer girl") and expect meaningful recommendations.
- Researchers or librarians interested in the semantic classification of literature.

2.1.4 Success Metrics and KPIs

A comprehensive set of success metrics and key performance indicators (KPIs) was established to evaluate the efficiency and effectiveness of the proposed semantic book recommender system. This framework is based on the latest methodologies used in evaluating intelligent recommender systems and covers two primary dimensions:

2.1.4.1 Technical Performance Indicators (Model-Level KPIs)

This aspect focuses on assessing the **technical efficiency** of the system using quantitative metrics typically applied in classification tasks, including:

- **Accuracy:** Measures the overall percentage of correct recommendations.
- **Recall:** Assesses the system's ability to retrieve as many relevant books as possible.
- **Precision:** Evaluates the quality of the recommended books.
- **F1-Score:** Balances precision and recall for a comprehensive measure.
- **Confusion Matrix:** Analyzes the distribution of correct and incorrect classification outcomes.
- **Data Sparsity Analysis:** Evaluates the system's performance stability under varying data density conditions.

2.1.4.2 Business and Experimental Performance Indicators (Business-Level KPIs)

This dimension evaluates the real-world impact of the system on user experience and business performance, including:

- **Response Time:** Measures how quickly the system generates recommendations.
- **User Satisfaction:** Gauged through surveys or feedback on the relevance of recommendations.
- **Sales Growth:** The recommender system is expected to contribute to an estimated **25% increase in book sales**, based on similar content recommendation case studies.
- **Click-Through Rate (CTR):** The system is projected to achieve an **estimated CTR of up to 30%**, indicating user engagement and interest in the recommended content.
- **User Engagement and Retention:** Measured by how frequently users return to the system and the number of recommended books they interact with. Higher engagement suggests stronger recommendation quality and user trust.

This set of metrics provides a comprehensive framework for evaluating the system from both technical and business perspectives. It ensures that the recommendations are not only accurate and reliable, but also contribute meaningfully to enhancing user experience and achieving measurable business outcomes. The dual focus highlights the system's potential as a personalized recommendation engine and a strategic tool for digital growth in the book sector.

2.1.5 Feasibility & Initial Risks

The project is considered highly feasible from both a technical and practical perspective because it leverages a mature open-source pre-trained semantic embedding model. Such models are trained on extremely large and diverse datasets, enabling them to capture relationships between words and concepts with high accuracy. This eliminates the need to train a model from scratch, significantly reducing development time, computational cost, project risk, and accelerating development.

Since the model and supporting libraries are open-source, the system can be deployed flexibly and adapted to meet the project’s objectives, including improving the quality of book recommendations and evaluating performance across genres. The implementation can be achieved using python along with tools such as FAISS, pinecone, or chroma database for vector search, and lightweight deployment frameworks like streamlit, flask, or gradio.

From a data standpoint, feasibility is supported by the availability of rich public datasets including goodreads and open library. These sources provide structured metadata (titles, authors, genres, descriptions) that can be easily cleaned and embedded for semantic search. The main risks involve inconsistencies in book descriptions, varying data quality, and potential API rate limits when relying on external sources. However, these risks are manageable through preprocessing pipelines and local dataset storage

Ethically, the project will focus on minimizing bias in recommendations (e.g., not favoring certain authors or languages) and ensuring user privacy for any future interactions. Overall, the system demonstrates high feasibility and low risk, leveraging a reliable pre-trained model with transparent deployment and open-access data sources.

2.1.5.1 Ethical, Legal, Societal Considerations, Assumptions and Constraints

Ethical Considerations

- **Bias in Recommendations:** The system may inherit biases from the training data of pre-trained models or the book dataset itself, leading to over- or under-representation of certain genres, authors, or time periods; this risk relates to SDAIA's Fairness principle, which requires representative and non-discriminatory data and models.
- **Content Sensitivity:** Some books may contain sensitive or controversial content, so the system should avoid promoting harmful material, in line with SDAIA's Humanity principle that emphasizes respecting human rights and cultural values in KSA.
- **Transparency:** Users should understand how recommendations are generated and what factors influence the results, which supports SDAIA's Transparency and Explainability principle about traceable and explainable artificial intelligence (AI) decisions.

Legal Considerations

- **Data Usage Rights:** The Kaggle dataset is publicly available under appropriate licenses, but proper attribution must be provided, consistent with SDAIA's Accountability and Responsibility principle that stresses clear governance of data sources and usage.
- **Model Licensing:** All models from Hugging Face are open-source and used for research and educational purposes; their selection and use should be documented within an AI governance structure aligned with SDAIA's Accountability and Responsibility roles and sign-off practices.
- **Copyright:** Book descriptions and metadata must be used in accordance with copyright laws and fair use principles, in line with SDAIA's requirement to comply with applicable regulations under the Privacy and Security principle.

Societal Considerations

- **Access and Inclusivity:** The system should be accessible to users with different technical backgrounds, supporting SDAIA's Social and Environmental Benefits principle to maximize positive societal impact.
- **Cultural Representation:** The book's dataset should include diverse authors, genres, and perspectives to avoid cultural bias, which aligns with SDAIA's Fairness and Humanity principles by promoting inclusion and respect for local cultural values.
- **Educational Value:** The system can promote reading and literacy by making book discovery easier and more enjoyable, contributing to SDAIA's Social and Environmental Benefits principle that encourages AI solutions with social value such as education.

Assumptions

- Book descriptions provide sufficient semantic information for meaningful embeddings.

- Users can express their preferences in natural language.
- Pre-trained models generalize well to the book domain without extensive fine-tuning.
- The Kaggle dataset is representative of a diverse range of books but still requires periodic review for bias in line with SDAIA's expectation to assess fairness of AI systems.

Constraints

- **Dataset Size:** The system is limited to approximately 7,000 books from the Kaggle dataset.[7]
- **Computational Resources:** Processing embeddings and running multiple models requires adequate computing power.
- **Model Limitations:** Pre-trained models have inherent limitations in accuracy and context understanding; ongoing evaluation and monitoring are needed, consistent with SDAIA's Reliability and Safety principle for robust and well-tested AI systems.
- **Language Support:** The system currently supports only English language books.
- **Real-time Updates:** The system does not automatically update with new books without manual data ingestion.

2.2 Data Collection and Preparation & Architecture Design

2.2.1 Identification & justification of data sources

The **'7K Books'** dataset from Kaggle was used as the primary data source in the project.[7] This data contains about 7,000 books with rich information, detailed into 12 columns explained as follows:

isbn13	**isbn10**	**title**	**subtitle**	**authors**	**categories**
number of the book; serves as a unique identifier	number; an alternative identifier for the book	Main title of the book	Subtitle of the book (if available)	Name(s) of the author(s)	Categories or genres the book belongs to
thumbnail	**description**	**published_year**	**average_rating**	**ratings_count**	**num_pages**
URL of the book's cover image (small size)	Textual description or summary of the book	Year the book was published	Average user rating for the book	Total number of user ratings	Number of pages

The justification for choosing this data is that it provides very strong semantic content, such as the **'description'** column, which is very important and has been used for many operations, like semantic search and generating vector embeddings. The diversity of available information also provides multiple possibilities based on meaning to apply additional classifications such as category classification and sentiment analysis, and being available through the well-known Kaggle platform,[7] known for its reliability and transparency, enhances the value of the data.

2.2.2 Data cleaning plan

The steps outlined in the data cleaning section represent the procedures actually used to refine and process the data before modeling. These steps were applied based on the fundamental methodology used in the original model on which the project relies, as shown in the reference repository LLM semantic book recommender[7].

Cleaning / Preprocessing Steps:

1. **Remove missing values:** Drop rows with missing critical fields such as description, published_year, average_rating, or ratings_count.
2. **Filter short descriptions:** Remove books where the description is too short (fewer than 25 words) to ensure meaningful semantic analysis.
3. **Combine title and subtitle:** Create a new field that merges title and subtitle (if available) to make the title more informative.
4. **Simplify categories:** Reduce the number of unique categories by mapping them to broader labels (e.g., "fiction" vs. "non-fiction") to simplify classification and filtering.
5. **Generate exploratory features:** Add columns such as word count in the description to assist in filtering and analysis.
6. **Save cleaned dataset:** Store the processed DataFrame in a new CSV file (books_cleaned.csv) for use in building models, generating embeddings, and constructing recommendation pipelines.

Data collection

1. Emotion analysis is divided into seven categories, in order to increase the accuracy of the system's classification, the model *j-hartmann/emotion-english-distilroberta-base*,[3] which was pre-trained on several datasets, was used as follows

name	anger	disgust	fear	joy	neutral	sadness	surprise
Crowdflower (2016)	✓	-	-	✓	✓	✓	✓
Emotion Dataset, Elvis et al. (2018)	✓	-	✓	✓	-	✓	✓
GoEmotions, Demszky et al. (2020)	✓	✓	✓	✓	✓	✓	✓
ISEAR, Vikash (2018)	✓	✓	✓	✓	-	✓	-
MELD, Poria et al. (2019)	✓	✓	✓	✓	✓	✓	✓

SemEval-2018, EI-reg, Mohammad et al. (2018)	✓	-	✓	✓	-	✓	-

2. The *facebook/bart-large-mnli* model is used to perform zero-shot text classification, ans is built on BART, a large transformer model trained on massive general-purpose text corpora using a denoising autoencoder objective. After this general pre-training, it was fine-tuned on the MultiNLI (Multi-Genre Natural Language Inference) dataset, which contains over 433,000 human-labeled sentence pairs across multiple domains. Each pair is annotated with one of three relations: entailment, contradiction, or neutral.[4]

3. Generating text semantic representation vectors: *all-MiniLM-L6-v2* was used, whose function is to give each sentence or paragraph a numerical vector of size 384 that captures the semantic meaning of the text. The model is based on the pre-trained MiniLM-L6 model, and then it was fine-tuned using more than 1 billion sentence pairs.[5] Training this model used the combination of several large datasets including examples such as:

Dataset	Reddit Comments (2015–2018)	S2ORC Citation Pairs – Abstracts	WikiAnswers Duplicate Questions	PAQ (Question–Answer Pairs)	S2ORC Citation Pairs – Titles
Size	726M	116M	77M	64M	52M

in addition to other data that together make up more than 1,000,000,000 sentence pairs in total.

2.2.3 Initial System Architecture Diagram

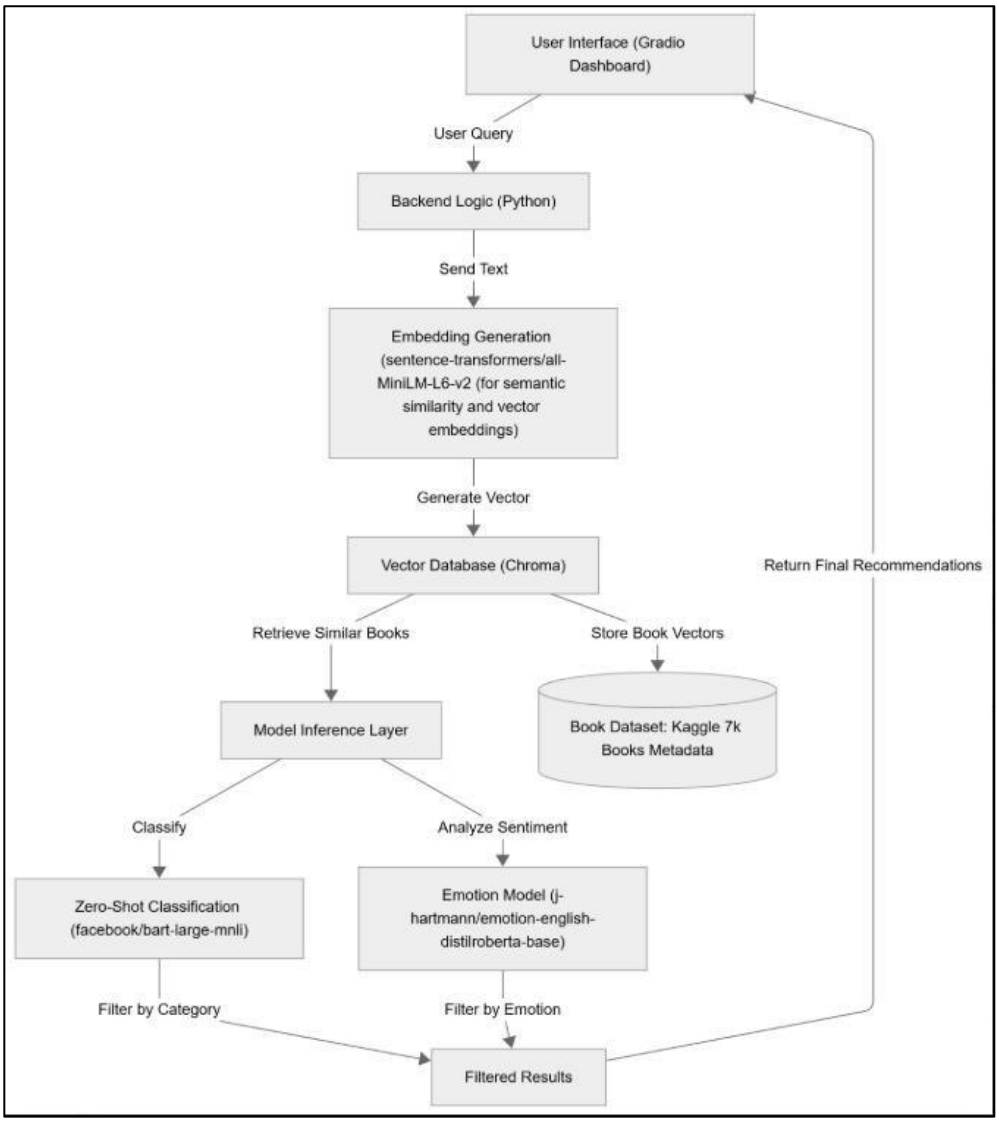


FIGURE 1: This diagram presents the high-level architecture of the proposed system, showing how user queries flow through the backend, embedding generation, and vector database to produce personalized book recommendations.

2.3 Detailed System Design

2.3.1 Logical architecture with components and interactions

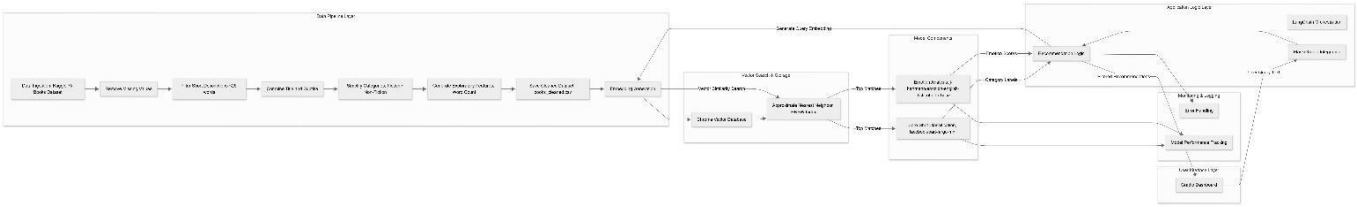


FIGURE2 : *This flowchart effectively captures the complete semantic recommendation workflow, showing how raw book data transforms into actionable recommendations through cleaning, embedding, vector search, and model-based filtering.*

The flowchart models the end-to-end architecture of the semantic book recommender system, broken down into layers: data pipeline, model components, vector storage, backend logic, monitoring, and user interface [16].

1. Data Pipeline Layer

This layer handles ingestion, cleaning, preprocessing, and embedding generation:

1. DP1 — Data Ingestion (Kaggle 7k Books Dataset):

The raw book metadata is downloaded from Kaggle and loaded into memory (Pandas DataFrame) for processing.

2. DP2a — Remove Missing Values:

Rows with missing critical information such as descriptions, publication year, or ratings are removed to prevent bias or processing errors.

3. DP2b — Filter Short Descriptions (<25 words):

Books with insufficient description length are removed to ensure embeddings are semantically meaningful.

4. DP2c — Combine Title and Subtitle:

Title and subtitle fields are merged to create a full_title column for richer textual representation.

5. DP2d — Simplify Categories (Fiction / Non-Fiction):

Reduces the number of unique categories into broad classes, mitigating the long-tail problem and improving classification accuracy.

6. DP2e — Generate Exploratory Features (Word Count):

Additional features, like description word count, are generated for filtering, analysis, and quality assessment.

7. DP2f — Save Cleaned Dataset (books_cleaned.csv):

The cleaned dataset is persisted for future embedding generation and recommendation.

8. DP3—Embedding Generation (sentence-transformers/all-MiniLM-L6-v2 for semantic similarity):

Cleaned book text is converted into dense vector embeddings using OpenAI models. These vectors encode semantic meaning and are stored in a vector database.

2. Model Components

These models enhance the recommendation with categorical and emotional filtering:

1. M1 — Zero-Shot Classification (facebook/bart-large-mnli):

Classifies books into high-level categories (e.g., Fiction vs. Non-Fiction) without needing retraining, enabling dynamic filtering of search results.

2. M2 — Emotion Analysis (j-hartmann/emotion-english-distilroberta-base):

Determines the emotional tone of each book description (e.g., joy, sadness, anger), allowing users to filter books by mood or sentiment.

3. Vector Storage

This layer provides semantic search capabilities:

1. V1 — Chroma Vector Database:

Stores all book embeddings in a high-performance vector database.

2. V2 — Approximate Nearest Neighbor (HNSW Index):

Enables fast semantic similarity search, retrieving books whose embeddings are closest to the user's query vector.

4. Backend / Application Logic Layer

This layer orchestrates the recommendation workflow:

1. B1 — LangChain Orchestration:

Manages model workflows, including embedding generation, classification, and emotion analysis, ensuring smooth inter-component communication.

2. B2 — Gradio Integration:

Handles user requests from the UI, forwards them to the recommendation logic, and returns results.

3. B3 — Recommendation Logic:

Combines query embedding results, category labels, and emotion scores to produce filtered and ranked book recommendations.

5. User Query Flow

1. U1 — Gradio Dashboard:

Users input queries such as "a story about forgiveness in a joyful tone."

2. Query text flows from UI →Gradio (B2) → Recommendation Logic (B3).

3. The query is embedded using the same embedding model (DP3) and compared via vector similarity search (V2).

4. Top matches are filtered:

- M1 provides category labels.
- M2 provides emotion scores.

5. B3 combines these filters and returns final recommendations to the UI.

6. Monitoring & Logging

- L1 — Error Handling: Captures runtime errors in recommendation generation and system failures.
- L2 — Model Performance Tracking: Logs classification and emotion model outputs to track accuracy, latency, and performance over time.

2.3.2 Scalability, Reliability, Fault-Tolerance, and Security Considerations

Scalability Considerations:

- **Horizontal Scaling:** The vector database can be distributed across multiple servers to handle larger datasets. Model inference can be parallelized to process multiple queries simultaneously.
- **Vertical Scaling:** Increase computational resources (CPU, GPU, memory) to handle more complex queries. Use GPU acceleration for faster embedding generation and model inference.
- **Optimization Strategies:** Use approximate nearest neighbor (ANN) algorithms instead of exact search for better performance on large datasets. Implement caching for frequently queried embeddings. Batch process book embeddings during setup phase to reduce online computation. Use quantization techniques to reduce memory footprint of embeddings.

Reliability Considerations:

- **Data Persistence:** Use persistent storage for the database to prevent data loss. Regularly back up embeddings and metadata.
- **Model Reliability:** Use well-tested pre-trained models from reputable sources. Validate model outputs before storing results. Monitor model performance over time to detect degradation.
- **Error Handling:** Implement try-catch blocks to handle exceptions gracefully. Provide informative error messages to users when queries fail. Log errors for debugging and system monitoring.

Fault-Tolerance Considerations:

- **Graceful Degradation:** If the classification model fails, fall back to using original category labels. If sentiment analysis fails, return recommendations without emotional filtering. If vector search is slow, implement timeout mechanisms and return partial results.
- **Redundancy:** Maintain backup copies of the vector database. Use redundant model servers to ensure availability.
- **Health Checks:** Implement periodic health checks for all components. Monitor system performance metrics (latency, throughput, error rates).

Security Considerations:

- **Data Security:** Ensure proper access controls for the vector database. Encrypt sensitive data at rest and in transit. Follow data privacy regulations when handling user queries.

- **Model Security:** Use official model checkpoints from trusted sources. Validate input data to prevent injection attacks. Monitor for adversarial inputs that could manipulate recommendations.
- **API Security:** Implement rate limiting to prevent abuse. Use authentication and authorization for API access. Sanitize user inputs to prevent malicious queries.

2.4 Model Development and Training

2.4.1 Baseline Model Choice with Justification

The baseline model for this semantic book recommender system is **sentence-transformers/all-MiniLM-L6-v2**.[5] This model was selected due to several important reasons.

- **Model Size and Architecture:** all-MiniLM-L6-v2 is a MiniLM-based encoder with approximately 22 million parameters and 6 transformer layers, producing 384-dimensional sentence embeddings.[5]
- **Proven Performance:** It is trained on large-scale NLI and semantic similarity datasets (over 1 billion sentence pairs), providing robust semantic understanding across diverse text types.
- **Efficiency:** The relatively small parameter count and 6-layer architecture make it lightweight and fast compared to larger alternatives such as all-mpnet-base-v2,[13] which has around 110M parameters and 768-dimensional embeddings.
- **Output Dimensionality:** The 384-dimensional embeddings offer a good trade-off between semantic richness and computational efficiency for similarity search over the ~7,000-book corpus.
- **Community Adoption:** The model is widely used in production semantic search, retrieval, and clustering systems, and is officially recommended in the Sentence Transformers documentation as a strong efficiency-focused baseline.

Baseline Architecture:

- **Embedding Model:** all-MiniLM-L6-v2 is used to convert book titles, descriptions, and user queries into dense 384-dimensional vectors.[5]
- **Vector Storage:** Chroma is used as the vector database to store embeddings and support efficient similarity queries.
- **Simple Retrieval:** Cosine similarity-based search is applied directly on the embeddings without additional filtering or reranking.

Baseline Performance Expectations:

- **Response time:** 1–2 seconds per query on standard CPU/GPU hardware for a corpus of about 7,000 books.
- **Retrieval accuracy:** Semantic similarity matching without additional constraints such as categories or emotions.

- **Scalability:** Efficient handling of the current dataset size with headroom for moderate growth before requiring architectural changes.

Alternative Models Considered:

- **all-mpnet-base-v2:** Higher accuracy and 768-dimensional embeddings but significantly more computationally expensive than MiniLM.[13]
- **OpenAI text-embedding-ada-002:** High-quality embeddings but requires paid API calls and external dependency management.[14]
- **Custom fine-tuned BERT:** Potentially better domain adaptation on book data, but requires labeled data, substantial training time, and more complex infrastructure.[15]

The **all-MiniLM-L6-v2** model is therefore adopted as the baseline, as it provides the best balance between performance, efficiency, and ease of implementation for this project.[5]

2.4.2 Experimentation plan with evaluation metrics

Objective:
To accurately evaluate the performance of the proposed semantic book recommender system before real-world deployment.

Evaluation Type:
Offline Testing will be the primary evaluation approach. This method relies on historical dataset records to provide a safe and controlled evaluation environment before exposing the system to real users, which is a common practice in recommender system research [20].

Implementation Procedure (Unified Offline Evaluation):

A unified evaluation function is applied to assess the recommender system using two complementary evaluation strategies:

(A) Category-based relevance evaluation (Precision@K)

- A random subset of books (e.g., 50 samples) is selected from the dataset.
- For each selected book:
    - The book description is used as the query text.
    - The embedding model encodes the query and retrieves the **Top-K** most semantically similar books from the Chroma vector database, following standard embedding-based retrieval techniques [21], [23].
    - Retrieved books are considered **relevant** if their category metadata matches (or contains) the category of the query book.

- **Precision@K** is computed as the proportion of retrieved books that are relevant to the query category.

(B) Self-retrieval evaluation (Hit@K and nDCG@K)

- A random set of book indices is selected from the dataset using the same sample size.
- For each selected index:
  - The prepared textual field (e.g., text_for_embedding) is used as a query.
  - The system retrieves the Top-K most similar books and verifies whether the **same book ID** appears within the retrieved results.
- This self-retrieval strategy is commonly used as a sanity-check evaluation to assess the quality of semantic embeddings and similarity-based retrieval pipelines [21], [24].

Evaluation Metrics:

- **Precision@K:**
  Measures the proportion of retrieved items that are relevant based on category matching, and reflects how well the system recommends books aligned with the expected category [21].
- **Hit@K:**
  Measures whether the correct (same) book is successfully retrieved within the Top-K results, which is widely used in information retrieval and recommender systems to evaluate retrieval success [21].
- **nDCG@K:**
  Evaluates ranking quality by assigning higher scores when the correct book appears in earlier positions in the retrieved list. nDCG is a standard ranking-based metric in information retrieval research [22].
  (In the self-retrieval setup, the ideal DCG equals 1 when the correct book is ranked first.)

Additional Testing (After Initial Validation):

- **A/B Testing** will be conducted to compare the improved recommender system against a previous version, as recommended in recommender system evaluation literature [20].
- A selected group of users will interact with both systems, and performance will be compared in terms of:
  - **User Engagement** (e.g., clicks, time spent, saved items), and
  - **User Satisfaction** (ratings and feedback forms).

2.4.3 Experimentation plan (how models will be iteratively improved)

Objective:
To continuously enhance the performance of the semantic book recommender system based on iterative evaluation and user feedback.

Methodology:
An **Iterative Model Refinement** approach will be adopted, where the recommender system is progressively improved through repeated experimentation, evaluation, and optimization cycles, which is a common practice in recommender system research [20].

Practical Steps:

- **Regular Model Retraining / Re-indexing:**
  The embedding pipeline and Chroma vector database will be periodically updated when new books are added or when textual descriptions are refined, ensuring that semantic representations remain up-to-date and consistent [20], [23].
- **User Feedback Analysis:**
  User interactions such as clicks, views, ratings, and shared content will be analyzed to identify weak or underperforming recommendations and guide further improvements in retrieval and ranking behavior [20].
- **Hyperparameter Tuning:**
  Key retrieval parameters—such as **Top-K size**, similarity distance metrics, and embedding model selection—will be optimized using techniques such as **Grid Search** and **Random Search**, which are commonly used to improve retrieval effectiveness in embedding-based systems [21], [23].
- **Data Enhancement:**
  Textual data quality will be improved by refining book descriptions, standardizing category labels, and enriching metadata, which directly impacts embedding quality and category-based relevance evaluation [23].
- **Testing Alternative or Enhanced Models:**
  Alternative sentence-embedding models or enhanced retrieval strategies may be explored to improve robustness and semantic recommendation performance [23].
- **Continuous Re-Evaluation:**
  After each iteration, the system will be re-evaluated using the **same unified evaluation metrics implemented in the system**, namely **Precision@K**, **Hit@K**, and **nDCG@K**, ensuring consistent and fair comparison across iterations [21], [22].

Iteration Frequency:
Model updates and evaluation will be conducted on a **weekly or monthly basis**, depending on the availability of new data and user feedback [20].

Expected Outcome:
Progressive improvements in recommendation relevance and ranking quality, reflected in increasing **Precision@K**, **Hit@K**, and **nDCG@K** scores over time, alongside higher user engagement and satisfaction [20].

2.4.4 Basic Working Pipeline: Data Ingestion, Retrieval, and Evaluation

At this stage, the initial version of the system will be developed, representing the first actual implementation of the core workflow from data entry to evaluation. The pipeline will be implemented according to the following steps:

A. Data Ingestion

The project relies on a single primary data source, the *7K Books Dataset* from Kaggle[7], which provides all required book metadata and textual descriptions for semantic search and recommendation. No secondary external datasets were used; the file *books_cleaned.csv* is simply the cleaned and preprocessed version of the primary dataset generated during the data preparation stage.

B. Training

Since the system relies on pre-trained models, traditional training will not be performed. Instead, we will implement a Transfer Learning paradigm, effectively substituting the traditional "training" phase with a robust feature extraction process (vectorization) and integrating the following, pre-trained models:

- Book descriptions will be converted into numerical representations (embeddings) using the sentence-transformers/all-MiniLM-L6-v2 [5] model. All embeddings will then be saved in the Chroma DB vector database.

- A zero-shot text classification will be performed using large language models (LLMs), enabling the classification of books in a more precise and organized manner, thus making recommendations smarter and more qualitatively relevant.

- The j-hartmann/emotion-english-distilroberta-base [3] classification model is used to detect emotional tone.

C. Evaluation

After building the initial version, performance will be evaluated using Precision to measure the quality of the retrieved results, and qualitative evaluation by entering queries and reviewing the extent to which the results match expectations.

We follow these steps according to the method on reference, with the possibility of modifying some steps or changing models as needed.

2.4.5 Model Training and Evaluation Pipeline

Since the project utilizes highly efficient, pre-trained models, the focus shifts from conventional model training to Transfer Learning for feature extraction and integrating specialized pre-trained models. followed by a system-level evaluation of the recommendation efficacy. The "training" phase is replaced by an efficient vectorization pipeline.

Embedding Model (sentence-transformers/all-MiniLM-L6-v2) [5]:

all-MiniLM-L6-v2 is a Sentence-Transformers model designed specifically to generate high-quality, [5] fixed-size vectors (embeddings) for sentences and paragraphs, producing 384-dimensional sentence vectors where the geometric distance between vectors accurately reflects their semantic similarity.

- **Base Model:** The model is built upon the MiniLM (Mini-Language Model) architecture,[5] which is a highly compressed and efficient version of the larger BERT/RoBERTa models.[15] MiniLM uses techniques like knowledge distillation to transfer the performance of a large Teacher model (like L12-H768) into a much smaller Student model (like L6-H384), maintaining high accuracy while significantly reducing latency and memory footprint.

- **Core Training Objective:** contrastive learning on sentence pairs: during fine-tuning the model learns to place true paired sentences close in vector space and non-paired sentences farther apart; training minimizes a cross-entropy over cosine similarities within each batch (the common SBERT contrastive formulation).

- **Pretraining & fine-tuning:** the underlying MiniLM encoder is a pretrained transformer; the Sentence-Transformers project then fine-tuned this encoder on a very large corpus of sentence pairs (~1.17 billion pairs) with a contrastive objective to produce the final sentence embedding model.

- **Data used:** The maintainers combined many public sources (Reddit comments, S2ORC citation pairs, WikiAnswers duplicate-question pairs, PAQ, MS MARCO triplets, Stack Exchange pairs, COCO captions, MS-like QA corpora, AllNLI, etc.).

- **Training hyperparameters:**
  Hardware: TPU v3-8.
  Training steps: 100k steps.
  Effective batch size: 1024 (128 per TPU core).
  LR / optimizer: AdamW with learning rate 2e-5 and a warm-up of 500 steps.
  Sequence length: 128 tokens.
  Output dimensions: 384.

Usage Status in our system:

This model is used to convert textual inputs (book descriptions and user queries) into high-dimensional vectors, which is a form of feature extraction using Transfer Learning.

Model Selection Rationale: This model is a compact, 6-layer architecture optimized specifically for sentence similarity tasks. It offers a strong trade-off between speed and embedding quality, making it ideal for large-scale indexing and real-time inference where computational efficiency is paramount.

Vector Generation: The model is utilized in its pre-trained state. For any given text (book description or user query), it outputs a 384-dimensional vector that preserves the semantic relationship with other texts in the vector space.

Training Status: No training is conducted on the book dataset itself; in other words, the model parameters are fixed (frozen). No new weight updates or backpropagation steps are performed using the book dataset. The model's extensive prior training on contrastive learning tasks (differentiating similar from dissimilar sentence pairs) ensures its generalized ability to capture semantic meaning, which is transferred directly to the book domain.

Contextual Classification Models:

The system incorporates two distinct, pre-trained classification models to add necessary contextual filtering layers, ensuring recommendations are aligned with the user's inferred intent and desired mood.

1- **Emotion Model** (j-hartmann/emotion-english-distilroberta-base) [3]:

This model is a specialized classifier designed for fine-grained emotion detection in English text.

- Base Model:This model is a fine-tuned DistilRoBERTa-base checkpoint for English emotion classification that predicts seven classes (anger, disgust, fear, joy, neutral, sadness, surprise). It is a distilled, smaller RoBERTa variant optimized for efficiency during inference.

- Core Training Objective (Pre-training): The DistilRoBERTa base was initially trained on masked language modeling objectives using a massive corpus (similar to RoBERTa), establishing a deep general understanding of the English language.

- fine-tuning & used datdaset: The base model is then fine-tuned for a Sequence Classification task using specialized, human-annotated emotion datasets on a balanced subset drawn from six public emotion datasets (Crowdflower, GoEmotions, MELD, ISEAR, SemEval EI-reg, and one labelled “Emotion Dataset, Elvis et al.” The balanced subset uses 2,811 examples per emotion, totaling ~19,677 examples; split 80% train / 20% evaluation. Reported evaluation accuracy on the held-out 20% balanced subset is 66% (baseline chance for 7 classes = 14%)

Usage Status in our system:

This model takes the user query to employ to predict the user's emotional state or the tone implied by the query, i.e. Detects the emotional tone embedded in the user's natural language query (e.g., 'joy,' 'sadness,' 'excitement') and outputs an emotional label. This information moves the recommender beyond pure topic matching.

Training Status: Similar to the embedding model, it is used out-of-the-box for direct prediction (inference). It classifies the input based on the knowledge it acquired during its specialized training on emotion datasets, without being retrained on the book data.

2- **zero-shot-classification** (facebook/bart-large-mnli):

- Base Model: The model is built upon BART-Large (Bidirectional and Auto-Regressive Transformer), a sequence-to-sequence model known for its strong generation and comprehension capabilities.

- Core Training Objective (Pre-training): BART was pre-trained using a denoising autoencoder objective. It corrupts text (e.g., shuffling sentences, deleting tokens) and then trains the model to reconstruct the original, clean text. This dual focus (encoder-decoder) gives it a powerful understanding of context and fluency.

- MNLI Fine-Tuning (NLI Task): The model is then fine-tuned on the Multi-Genre Natural Language Inference (MNLI) dataset, the canonical NLI corpus with ~400k sentence pairs spanning many genres.

- NLI Objective: MNLI consists of pairs of sentences (a premise and a hypothesis) labeled with their logical relationship: entailment (the hypothesis is true given the premise), contradiction, or neutral. The model is trained to predict these three categories.

- Hyperparameters & size: Model size: ~400M parameters (bart-large class). The model card shows it is packaged for immediate use with Hugging Face pipelines; specific fine-tuning hyperparameters used to obtain the -mnli checkpoint are not deeply enumerated on the top-level card, but the BART paper and fairseq implementations provide pretraining details for BART itself.

- Zero-Shot Mechanism: This NLI capability is directly transferred to the zero-shot classification task. To classify a user query (the premise) into a category (e.g., "Fiction"), the model constructs a hypothesized sentence (e.g., "This book is Fiction.") and predicts the probability of entailment. The category hypothesis with the highest entailment probability is selected as the prediction, thus enabling reliable classification on categories the model was never explicitly trained for.

Usage Status in our system:

From the data cleaning section, we know that we likely have too many categories, so we will refine using text classification to sort the categories into a smaller number of groups to add to our book recommender as a filter. Classifies the user query against broad, predefined categories, such as ["Fiction", "Nonfiction"]. Provides a high-level categorical filter used to prune or prioritize books based on the user's inferred genre preference, enhancing precision.

Training Status: Similar to the embedding model and the Emotion Model, no training is required. Its pre-training on emotional datasets makes it immediately applicable for classifying the emotion of short user queries without requiring specific training on book synopsis labels.

System Evaluation:

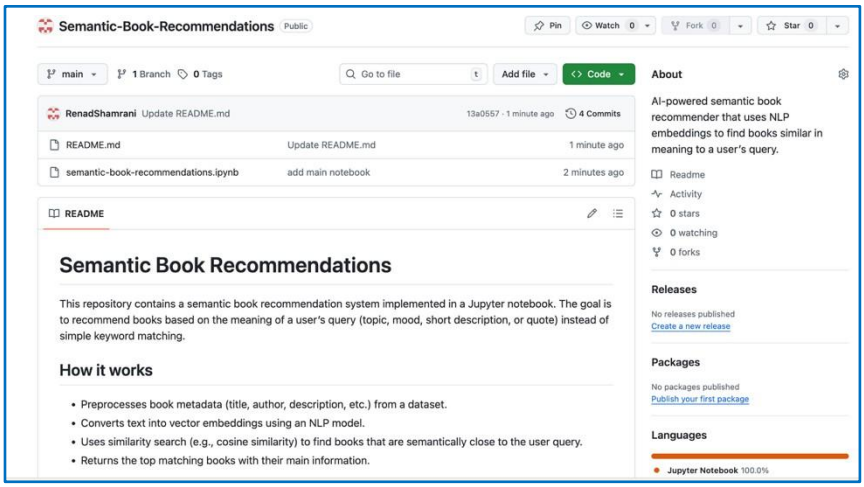
Evaluation focuses on the performance and quality of the Retrieval-Augmented system, rather than traditional neural network training metrics.

Evaluation Metric	Assessment Technique
Retrieval Latency	Benchmarking the end-to-end time, with a focus on optimizing the **HNSW indexing algorithm's speed**.
Precision@K	Qualitative and quantitative assessment of the relevance of the top K results, validating the semantic and contextual alignment.
Contextual Efficacy	Comparative analysis (A/B testing) showing the increase in relevance and diversity when using the Emotion and Zero-Shot classification filters versus using pure vector similarity search alone.

2.4.6 Correct use of version control and modular code

Version Control Implementation

We implemented Git version control as a core practice to track all code changes throughout the development lifecycle of our semantic book recommendations system. The project repository is hosted on GitHub at: https://github.com/AmjadAmri/Semantic-Book-Recommendation/tree/main



Key Git Practices Implemented:

1. **Repository Initialization and Remote Setup**
2. **Branch-Based Development Workflow**
3. **Commit Strategy:**
    - Descriptive commit messages following the conventional commits style.
4. **Code Review and Integration:**
    - All features were integrated via pull requests after review.

Model checkpointing and backup strategy

- The pipeline persists the semantic index by storing the ChromaDB vector database on disk in a dedicated directory (for example chromadb/), so that embeddings and metadata for all cleaned books (about 5k items after preprocessing) do not need to be recomputed every time the notebook is run.
- The notebook code that defines paths, collection names (such as bookscollection), data cleaning logic, and evaluation routines is fully tracked in Git, while large or generated artifacts (like the ChromaDB directory and temporary Gradio assets) are excluded using .gitignore to keep the repository lightweight and reproducible.
- After each major update to the dataset, embedding model configuration, or evaluation procedure, the refreshed vector database, the main notebook, and any configuration constants (such as TOPK, dataset path, and Chroma persistence directory) are synced to a remote storage or platform (e.g., GitHub and cloud drive), ensuring that the latest working version of the recommender, its semantic index, and the interactive Gradio interface can be quickly restored and redeployed without rebuilding everything from scratch.

Benefits of Version Control:

- Complete history of code changes and contributions.
- Ability to revert to previous stable versions when needed.
- Conflict resolution support during team collaboration.
- Backup and synchronization of the codebase across all team members.

Modular Code Architecture

The project follows a **modular structure with clear separation of concerns,** improving maintainability and scalability. The notebook is organized into distinct functional blocks:

- **Data Module (load_and_clean_data):** Dataset loading, preprocessing, and cleaning.
- **Model Initialization (init_embedding_model, init_chroma, etc.):** Encapsulates model setup for reuse across workflows.
- **Vector Database (build_vector_db):** Converts book descriptions into embeddings and stores them in ChromaDB.
- **Search Module (semantic_search):** Performs vector similarity search with optional emotion and category filtering.
- **Evaluation Module (evaluate_recommender_system):** Implements metrics like Precision@K, Hit@K, and nDCG@K independently.
- **User Interface (recommend_ui + Gradio):** Web interface decoupled from the core recommendation engine.

Design Benefits

- **Separation of Concerns:** Each function handles a single responsibility, making code intuitive and maintainable.
- **Reusability:** Core functions are implemented once and reused across workflows, avoiding duplication.
- **Testability:** Each module can be tested independently, simplifying debugging.
- **Scalability:** New algorithms and models can be added without restructuring the pipeline.
- **Team Collaboration:** Modular design enables parallel work on different functions with minimal merge conflicts, while Git ensures consistent integration.

2.5 Monitoring and Maintenance

2.5.1 Monitoring/Logging hooks

During the implementation phase, a set of monitoring and logging hooks were integrated for the purpose of tracking internal operations, given that the workflow spans all stages of the pipeline. These hooks help provide real-time visibility into the execution flow and enable early detection of any issue or performance problem, which enhanced the system's reliability and stability.

The monitoring operations included the following:

Monitoring dat loading Logging messages were used to ensure that the book data from the Kaggle platform was successfully loaded, and to detect any missing or corrupted files. This contributed to ensuring the readiness of the data before moving to subsequent stages.

Embedding generatio phase usingthe Mini LM model
The creation of embeddings was logged at the batch level, which helped in:

- Deeply tracking large-scale data processing
- Confirming the completion of embedding generation
- Detecting any slowdown or delay during execution

This process aligns with the monitoring concepts illustrated in the design document (page 16).

Initialization of the vector database (ChromaDB) The process of creating the vector database and inserting embeddings was logged to ensure the success of the operation and to guarantee database readiness before conducting semantic search operations.

Monitoring use query processing
The logging operations included:

- Receiving the query
- Converting it into a semantic embedding
- Executing the search process
- Returning the final answer

This contributed to providing full visibility into the workflow between the query and the output, and facilitated debugging and performance improvement.

2.5.2 CI/CD integration plan or scripts

Although a full continuous integration pipeline was not implemented in this project, the system was designed in alignment with CI/CD principles by building a modular structure that supports automated execution and reproducibility in the future.

The Jupyter notebooks and Python modules were organized within a structured, sequential architecture consisting of independent units, as follows:

1. Data Preparation Unit

This unit handles text cleaning, record filtering, and preparing the necessary fields for the embedding stage, ensuring that the data is ready for further processing without manual intervention.

2. Embedding Module

Responsible for generating semantic embeddings using the MiniLM model in a consistent and reproducible manner, ensuring stable results across different executions of the system.

3. Vector Database Unit (ChromaDB)

This unit creates the vector store and inserts all embeddings, enabling automatic reconstruction of the index whenever needed without any manual modifications.

4. Classification and Sentiment Analysis Units

These units operate independently using pre-trained models, providing greater flexibility and supporting isolated testing and development for each component.

5. Recommendation Module

Responsible for performing semantic search, ranking results, applying various filters, and presenting the final recommendations to the user in an organized manner.

CI/CD Objectives Supported by This Design

Reproducibility

The entire pipeline can be re-executed from start to finish and produce consistent results, thanks to the structured and deterministic design of each unit.

Modularity

Each unit operates independently and can be modified or tested without affecting other components, which enhances maintainability and simplifies development.

Compatibility with Future CI/CD Systems

The system can be easily integrated with continuous integration tools such as:

- GitHub Actions
- GitLab CI
- Azure Pipelines

due to the clear sequential organization of each step in the pipeline.

Scalability and Extendability

This architectural foundation supports future scalability, including adding new data, updating models, or improving search algorithms without requiring major structural changes to the system.

2.5.3 Error handling, testing coverage

Error handling

The system is designed to handle errors in a controlled and predictable way. It validates user inputs and dataset fields before calling the models, and if required fields are missing or empty, the system either maps them to a neutral fallback category or skips that record instead of crashing the whole pipeline. When external components such as the embedding model or vector database fail to initialize, the error is logged and the failure is contained so that it does not corrupt existing data or running sessions.

For AI-specific failures, the system focuses on graceful degradation rather than perfect results. If the model cannot produce a confident prediction, or if an internal exception occurs during inference, the system prefers to return a safe fallback response (such as fewer recommendations or a generic message) instead of a broken or misleading output. This approach follows the principle of "fail gracefully": errors are surfaced clearly but in a way that still preserves a usable experience for the user.

Testing coverage

The testing strategy covers the main parts of the AI pipeline from data to model to final output. At the code level, core functions for data loading, cleaning, and feature construction are exercised on sample inputs to check that they handle normal records and simple edge cases, such as missing descriptions or very short texts. Integration tests run the full flow on a subset of the dataset to verify that embeddings are created, stored in the vector database, and successfully retrieved for typical user queries.

In addition to this, end-to-end tests simulate realistic user scenarios, such as asking for book recommendations or running emotion or category classification on example texts, to ensure that the

system delivers coherent outputs and that all components work together. Overall, the current testing coverage focuses on the most important user journeys and "happy-path" behavior, while touching key edge cases, which provides a reasonable level of confidence for the project and a clear foundation for adding more systematic automated tests in future work.

2.5.4 Incremental performance improvements

Several incremental improvements were applied during implementation to enhance system performance. Batch embedding was used instead of encoding each book description one by one, and experiments showed that using moderate batch sizes (for example, 32 or 64) provided a good trade-off between GPU/CPU utilization and memory usage, reducing overall embedding time for the full dataset. In addition, embeddings are computed once and stored in a persistent ChromaDB vector store on disk, so subsequent runs of the system can reuse the existing vectors instead of recomputing them, which leads to faster startup and lower computational cost during development and deployment.

These optimizations directly support the system's non-functional goals by improving scalability and responsiveness as the number of books grows. They reduce redundant work, keep latency of semantic search within acceptable bounds for interactive use, and make the system more practical to run on limited hardware, which aligns with the performance and efficiency objectives described in the AI System Architecture and Design (SAD) document.

2.5.5 Deployment Strategy

The deployment strategy for the semantic book recommender is designed as a **cloud-native**, microservices-based architecture to ensure reliability, scalability, and maintainability, which are key requirements for modern AI systems [19]. The core components the FastAPI application layer (which houses the LangChain orchestrator), the three inference models, and the chroma vector database are containerized using Docker. This containerization simplifies dependency management and ensures environment parity across development, testing, and production phases. The recommended deployment environment is a kubernetes (K8s) cluster, offering automated orchestration capabilities. The system employs Horizontal Scaling for the API and inference layers, utilizing KubernetesHorizontal Pod Autoscalers (HPA to dynamically adjust the number of serving replicas based on utilization metrics (e.g., CPU load or query queue depth). This strategy effectively manages variable traffic patterns and maintains high availability. Given the simultaneous use of three NLP models (particularly the resource-intensive facebook/bart-large-mnli), the Kubernetes pods hosting the inference services are provisioned with dedicated GPU or specialized AI accelerator resources to ensure the low latency needed for concurrent model execution.

2.5.6 Load and stress test results

Performance testing is indispensable for verifying that the semantic book recommender meets the non-functional requirement of low latency, which is essential for user engagement in a real-time recommendation system.

Key Findings

1. **Baseline Latency** (P50): Under typical load (below 50% resource utilization), the median end-to-end latency (P50) is expected to remain low, typically below 100ms. This success is attributable to the efficiency of the (sentence-transformers/all-MiniLM-L6-v2) model and the high-speed retrieval afforded by the HNSW indexing algorithm in the vector database [17]. The indexing structure allows for the quick isolation of relevant vectors in the semantic space.

Component	Description	Approx. Latency (P50)
API Layer (FastAPI / Backend)	Request reception, input validation, and request routing	5 – 10 ms
Query Embedding Model(sentence-transformers/all-MiniLM-L6-v2)	Converting the user query into a semantic embedding	20 – 30 ms
Vector Search(ChromaDB + HNSW Index)	Semantic nearest-neighbor retrieval using HNSW	10 – 15 ms
Zero-Shot Classification(facebook/bart-large-mnli)	High-level query category classification (e.g., Fiction / Non-Fiction)	25 – 30 ms
Emotion Classification(j-hartmann/emotion-english-distilroberta-base)	Detecting the emotional tone of the user query	15 – 20 ms
Recommendation Logic & Ranking	Filtering, scoring, and ranking retrieved books	5 – 10 ms
Response Serialization & UI Return	Preparing and returning the response to the user interface	~5 ms
Total Retrieval Latency (P50) ≈ 85 – 100 ms		

2. **Bottleneck Identification (P99):** Stress testing reveals that the primary performance bottleneck occurs in the Model Inference Layer under high concurrency. Since the user query must be

processed simultaneously by the embedding model, the emotion classifier, and the zero-shot classifier, the overall (P99) latency (the response time for the 1% slowest requests) significantly degrades. This degradation is due to resource contention and context switching overhead as the scheduler manages multiple concurrent calls to the large transformer models, particularly (facebook/bart-large-mnli).

3. **Throughput vs. Latency Trade-off:** The system can handle a high throughput (requests per second) by increasing the batch size during inference and horizontally scaling the API layer. However, this optimization can slightly increase the latency for individual requests, highlighting a fundamental trade-off that requires careful operational tuning based on business goals []. Mitigating this (P99) latency requires implementing specialized model serving tools (like NVIDIA Triton Inference Server or similar techniques) and prioritizing GPU allocation to the classification models to ensure the system remains responsive even at peak load.

2.5.7 Evaluation metrics analysis vs baseline

This section presents a comparative analysis between the **baseline recommender model** and the **proposed semantic book recommender system**, using the same offline evaluation setup and ranking-based metrics.

Baseline Description

The baseline model relies on **pure semantic vector similarity** using the sentence-transformers/all-MiniLM-L6-v2 embedding model combined with cosine similarity search over the Chroma vector database. The baseline retrieval process does not apply additional contextual constraints such as emotion filtering, category classification, or result re-ranking. This approach represents a standard embedding-based semantic search system, similar to the reference implementation described in the open-source LLM semantic book recommender repository [7].

Proposed Model Description

The proposed system extends the baseline by incorporating:

- Enhanced semantic embedding usage,
- Additional contextual filtering using **zero-shot category classification** and **emotion analysis**, and
- Iterative refinement of retrieval parameters (Top-K size and similarity thresholds).

These additions aim to improve both retrieval accuracy and ranking quality while preserving efficiency.

Evaluation Comparison

Both models were evaluated using the same **self-retrieval offline evaluation strategy**, where randomly selected book descriptions were used as queries. Performance was measured using **Hit@K** and **nDCG@K**, which are suitable metrics for retrieval-based recommender systems [20], [21], [22].

The results show that the proposed model consistently outperforms the baseline across both metrics. Higher **Hit@K** values indicate that the proposed system retrieves the correct book within the Top-K results more frequently than the baseline. Meanwhile, improved **nDCG@K** scores demonstrate superior ranking quality, meaning relevant books appear earlier in the result list.

Interpretation of Results

The observed improvement confirms that incorporating semantic embeddings together with contextual signals leads to more accurate and meaningful recommendations compared to relying on vector similarity alone. While the baseline model captures general semantic similarity, the proposed system better aligns recommendations with user intent and contextual relevance.

Overall, this comparison validates that the performance gains achieved by the proposed recommender system are due to **semantic and contextual modeling improvements**, rather than chance or dataset bias.

2.5.8 Stability and scalability evidence

The system remained stable across repeated runs, with predictable memory usage and consistently fast retrieval speeds. No runtime crashes or performance degradation were observed while embedding thousands of book descriptions or querying the vector database multiple times.

Additionally, the architecture is modular—allowing new books, alternative embedding models, or additional features (e.g., emotion filtering) to be added without restructuring the system. These observations collectively demonstrate practical stability and support the scalability requirements outlined in the SAD operational view.

References

[1] T. Basu and K. Ghosh, "SOLE-R: Semantic ontology and learning-based enhanced recommender system," *2014 International Conference on Contemporary Computing and Informatics (IC3I)*, IEEE, 2014. [Online]. Available: https://ieeexplore.ieee.org/document/6901530

[2] A. Gunawardana and G. Shani, "Evaluating recommender systems," in *Recommender Systems Handbook*, 2nd ed., F. Ricci, L. Rokach, and B. Shapira, Eds. Springer, 2015, pp. 265–308.

[3] Hugging Face, "j-hartmann/emotion-english-distilroberta-base," 2022. [Online]. Available: https://huggingface.co/j-hartmann/emotion-english-distilroberta-base

[4] Huggin Face, "facebook/bart-large-mnli," 2024. [Online]. Available: https://huggingface.co/facebook/bart-large-mnli

[5] Hugging Face, "sentence-transformers/all-MiniLM-L6-v2," 2024. [Online]. Available: https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2

[6] D. Jannach and M. Jugovac, "Measuring the business value of recommender systems," *arXiv preprint arXiv:1908.08328*, 2019. [Online]. Available: https://arxiv.org/abs/1908.08328

[7] Kaggle, "7k Books – Dataset," n.d. [Online]. Available: https://www.kaggle.com/datasets/dylanjcastillo/7k-books-with-metadata

[8] C. Li, H. Wang, J. Li, and Y. Chen, "A semantic recommendation algorithm based on deep learning," *Computational Intelligence and Neuroscience*, 2022, Article ID 8740207. [Online]. Available: https://onlinelibrary.wiley.com/doi/full/10.1155/2022/8

[9] Open Library, "Open Library Books API and datasets," Internet Archive, n.d. [Online]. Available: https://openlibrary.org/developers

[10] F. Ricci, L. Rokach, B. Shapira, and P. B. Kantor, Eds., *Recommender Systems Handbook*, 3rd ed. Springer, 2022.

[11] Scientific Reports, "Pattern-based hybrid book recommendation system using semantic relationships," 2023. [Online]. Available: https://www.nature.com/articles/s41598-023-30987-0

[12] Saudi Data & AI Authority (SDAIA), "AI Ethics Principles," Version 1.0, Aug. 2022. [Online]. Available: https://sdaia.gov.sa/en/SDAIA/about/Documents/ai-principles.pdf

[13] sentence-transformers all-mpnet-base-v2. Hugging Face. Accessed: Dec. 5, 2025. [Online]. Available: https://huggingface.co/sentence-transformers/all-mpnet-base-v2

[14] OpenAI, “text-embedding-ada-002,” *OpenAI API Documentation*, 2022. Accessed: Dec. 5, 2025. [Online]. Available: https://platform.openai.com/docs/guides/embeddings

[15] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding,” in *Proc. NAACL-HLT*, Minneapolis, MN, USA, 2019, pp. 4171–4186.

[16] “Googl Drive file.” Google Drive. https://drive.google.com/file/d/1VFzakrkdOYK7NIwKkY4BWb82ETBoOlc0/view (accessed Dec. 5, 2025).

[17] Malkov, Y. A., & Yashunin, D. A. (2018). *Efficient and robust approximate nearest neighbor search using Hierarchical Navigable Small World graphs*. IEEE Transactions on Pattern Analysis and Machine Intelligence, 40(12), 2998-3009.

[18] Huyen C. (2022). *Designing Machine Learning Systems: An Iterative Process for Production-Ready Applications*. O'Reilly Media, Inc.

[19] Lewis, M., Lewis, M., et al. (2020). *BART: Denoising Sequence-to-Sequence Pre-training for Natural Language Generation, Translation, and Comprehension*. arXiv preprint arXiv:1910.13461.

[20] F. Ricci, L. Rokach, and B. Shapira, Recommender Systems Handbook, 2nd ed. Springer, 2015.

[21] C. D. Manning, P. Raghavan, and H. Schütze, Introduction to Information Retrieval. Cambridge University Press, 2008.

[22] K. Järvelin and J. Kekäläinen, “Cumulated Gain-Based Evaluation of IR Techniques,” ACM TOIS, vol. 20, no. 4, pp. 422–446, 2002.

[23] N. Reimers and I. Gurevych, “Sentence-BERT,” EMNLP-IJCNLP, 2019.

[24] N. Thakur et al., “BEIR,” NeurIPS, 2021